Sending your firmware and logs to the repo

A step-by-step guide to getting your work onto its own GitHub branch, with Claude doing most of it

Repository: **github.com/Tangineering/DC-Balancer-Board-Teensy**

What this does

You have firmware versions in separate folders, Raspberry Pi data logs, and the analysis scripts you use on those logs. They need to go into the project's GitHub repository. You won't be changing the main copy. Your files go onto a **separate branch**: a parallel copy of the repository that only holds your additions. I review the branch and merge it into **main**. Nothing you do here can break main.

Claude does the git work. You install two programs, paste three prompts, answer Claude's questions and sign in to GitHub once. Plan on 30 to 60 minutes, mostly waiting on the download.

The words you'll see

Term	Meaning
Repository (repo)	The project's folder on GitHub, with its full history.
Clone	Download a full working copy of the repo to your computer. This is a **new** folder, kept separate from your current folders.
Branch	A named, separate line of changes. Yours will be called something like `phd/yourname-2026-11`.
Commit	A saved snapshot of changes with a short description. Each firmware version becomes one commit, and the Pi logs and the analysis scripts get one commit each.
Push	Upload your commits from your computer to GitHub.
Pull request (PR)	A request on GitHub asking me to review the branch and merge it into main.

The rules

- **Leave your existing folders alone.** Claude only copies from them. It never moves, renames or deletes anything in them.
- **Never push to main.** All work goes on your own branch. The prompts tell Claude this too.
- **Claude only packages files.** It must not edit, fix or reformat any firmware. The repo includes instructions for Claude about firmware work (a file called CLAUDE.md), and Claude will read them. The prompts tell it they don't apply to this task.

Part 1 — One-time setup (about 15 minutes)

Step 1. Accept the GitHub invitation

You've been added to the repo as a collaborator. Open the invitation email from GitHub (or go to `github.com/Tangineering/DC-Balancer-Board-Teensy` while signed in) and click **Accept invitation**. You can't push until you accept.

Step 2. Install Git

- **Windows:** download from `git-scm.com/downloads` and run the installer. Accept every default. The defaults include the Git Credential Manager, which handles GitHub sign-in later.
- **Mac:** open Terminal, type `git --version` and press Enter. If Git is missing, the Mac offers to install it (the Command Line Developer Tools). Click Install.

Step 3. Get Claude Code

The regular Claude chat can't touch files on your computer. You need **Claude Code**, the version of Claude that works in a folder on your machine:

- Install the **Claude desktop app** from `claude.ai/download` and sign in. Open the **Code** tab. When it asks for a folder, choose your **Documents** folder (the next step clones the repo there).
- Or, if you're comfortable in a terminal: install Claude Code from `claude.com/claude-code`, then run `claude` from your Documents folder.
- Claude will ask permission before it runs commands or writes files. Read each request and approve it. That's normal.

> **No Claude Code?** (It needs a Pro or Max plan, or similar.) See the appendix at the end for a slower fallback that only uses the GitHub website.

Part 2 — Prompt 1: download the repo

Paste this into Claude Code and fill in the three bracketed values. Use the email address on your GitHub account.

PROMPT 1 — copy everything in this box

```
I'm new to Git and GitHub. Please do the following one-time setup for me, explaining
each step in one plain sentence as you go.

1. Check that git is installed (git --version). If it is not, stop and tell me.
2. Set my git identity, but only if it is not already set:
     git config --global user.name  "[MY FULL NAME]"
     git config --global user.email "[EMAIL ON MY GITHUB ACCOUNT]"
   Also, only on Windows, run:  git config --global core.autocrlf true
3. In my Documents folder, create a folder named GitHub (if missing) and clone:
     https://github.com/Tangineering/DC-Balancer-Board-Teensy
   into Documents/GitHub/DC-Balancer-Board-Teensy. It is about 1.5 GB, so it
   takes a while. If git asks me to sign in to GitHub, tell me to complete the
   sign-in in the browser window that opens. My GitHub username is [MY USERNAME].
4. When the clone finishes, show me the latest 3 commits on main
   (git log --oneline -3) and confirm the clone succeeded.
```

```
Do not change any files in the repository. This step is download only.
```

If a browser window opens asking you to sign in to GitHub or authorize Git Credential Manager, sign in and approve it. Then **close this Claude session and start a new one** with the folder set to `Documents/GitHub/DC-Balancer-Board-Teensy`. (Desktop app: pick that folder for the new session. Terminal: `cd` into it and run `claude`.)

Part 3 — Prompt 2: look first, change nothing

This prompt has Claude look at your folders and write a plan. It doesn't change anything yet. Fill in the paths to your folders. The easiest way to get a path: on Windows, Shift + right-click the folder and choose **Copy as path**. On a Mac, right-click it, hold Option and choose **Copy … as Pathname**.

PROMPT 2 — copy everything in this box

```
I'm the PhD student on this project. I have firmware versions, Raspberry Pi
data logs, and Python analysis scripts (written with Claude) for those Pi logs.
They live OUTSIDE this repository, in my own folders, and I want them committed
on a NEW BRANCH so my supervisor can review and merge them. I'm new to Git, so
please do the Git work for me.

IMPORTANT: this task is ONLY packaging my files into Git. The CLAUDE.md firmware
instructions in this repo do NOT apply to this task. Do not edit, fix, reformat,
review, or "reconcile" any firmware or log file. Copy my files exactly,
byte-for-byte. Never modify, move, or delete anything in my own folders. Never
commit or push to main.

My folders:
  Firmware versions (one sub-folder per version): [PATH TO FIRMWARE FOLDER]
  Raspberry Pi data logs:                         [PATH TO PI LOGS FOLDER]
  My Pi-log analysis scripts:                     [PATH TO SCRIPTS FOLDER]
  (Anything else I want included, or "nothing":  [PATH OR "nothing"])
The firmware version I STARTED from, if I know it: [e.g. "fw v28", or "not sure"]

For this step, do NOT change anything. Only look and report:

1. Run git fetch origin and git status. Confirm I'm on main with no local changes.
2. In this repo, the firmware sketch is teensy_controller/teensy_controller.ino
   (plus the .h files in that folder), Teensy SD-card logs (.BLG) are in logs/,
   and analysis tools are in tools/. My logs come from the Raspberry Pi, so they
   are a DIFFERENT format from the .BLG files. Don't convert them, and don't feed
   them to the repo's BLG decoder.
3. List my firmware version folders. For each one, find its .ino and .h files,
   its FW_VERSION #define, and its BENCH_TEST / HIL_SIM #define values. Work out
   the order, oldest to newest (by version number, then file dates). Show a table.
4. Find my STARTING POINT: compare my oldest version's teensy_controller.ino
   against the history of teensy_controller/teensy_controller.ino in this repo
   (git log --follow, and git diff against candidate commits). Tell me which repo
   commit is identical or closest, and how big the difference is. The branch will
   start from that commit, so my changes don't accidentally undo newer work on main.
5. List my Pi log files with sizes and file types. Flag (a) any file over 50 MB
   (GitHub rejects files over 100 MB), and (b) anything that is not data:
   build output (.hex, .elf, build folders), .zip files, temp or IDE files, or
   anything that looks private.
6. List my analysis scripts. For each one, say in one line what it does and which
   Python packages it imports. Flag (a) hard-coded paths into my own folders
   (just report them; do NOT edit the scripts), (b) anything that looks like a
   password, API key or token, and (c) generated output sitting next to the
   scripts (plots, CSVs, __pycache__, .ipynb_checkpoints, virtual environments).
   Ask me which generated output is worth keeping.
```

```
7. Propose a plan. Do not carry it out yet:
   - branch name: phd/[MY FIRST NAME]-<today's date as YYYY-MM-DD>
   - one commit per firmware version, oldest first, each copying that version's
     .ino and .h files over teensy_controller/ (only files that exist in that
     version), with a message like "PhD fw vNN: <short summary of what changed,
     from the diff>"
   - one commit putting my Pi logs in logs/pi/from_[MY FIRST NAME]_<date>/,
     keeping my original file names and sub-folder structure
   - one commit putting my analysis scripts in tools/pi_log_analysis/, with a
     short README.md that YOU write: what each script does, its inputs/outputs,
     the packages it needs (also as requirements.txt), and how to run it on the
     logs folder above. The README is the only new file you write; the scripts
     themselves are copied unchanged.
   - anything you'd leave out, and why
Then stop and wait for me to confirm or correct the plan.
```

Read Claude's report before you go on. Check these things:

- **Version order:** are the versions in the right order? Correct Claude if not.
- **Commit summaries:** these come from the code differences, and you know what you actually changed. Give Claude a sentence for each version. That's the most useful thing you can add for my review.
- **Starting point:** if Claude says your oldest version matches no repo commit exactly, that's fine. Say "use the closest one." Don't merge or combine anything yourself; I'll handle that.
- **Flagged files:** tell Claude to leave out anything that shouldn't be shared. Never commit a password or API key, even on a branch.

Part 4 — Prompt 3: commit and push

Once the plan looks right, paste this in the same session:

PROMPT 3 — copy everything in this box

```
The plan is approved, with these corrections: [YOUR CORRECTIONS, or "none"].
What changed in each version, in my own words (use these in the commit messages):
  [vNN: ...]
  [vNN: ...]

Now carry it out:
1. Create the branch from the starting-point commit you identified
   (git switch -c <branch> <commit>). Never commit to main.
2. For each firmware version, oldest first: copy its files into
   teensy_controller/, run git status and git diff --stat to confirm only the
   expected files changed, then commit with the agreed message. Include
   FW_VERSION and the BENCH_TEST / HIL_SIM values in each message.
3. Copy the Pi logs into logs/pi/from_[MY FIRST NAME]_<date>/ and commit them as
   "PhD Pi logs: <count> files, <date range>".
4. Copy the analysis scripts into tools/pi_log_analysis/, add the README.md and
   requirements.txt, and commit as "PhD Pi-log analysis scripts".
   For both commits: skip anything we agreed to leave out, and tell me if
   .gitignore excludes any file.
5. Verify the copies: compare a checksum of every committed file with the
   original in my folders, and report any mismatch.
6. Show me git log --oneline for the branch, then push it:
     git push -u origin <branch>
   If the push fails because of sign-in, explain what to do. If it fails for a
   network reason, retry a couple of times.
7. Finish with a summary I can paste into an email to my supervisor: the
```

```
   branch name, the starting commit, one line per commit, and the link GitHub
   printed for opening a pull request.

Again: copy my files exactly. Do not edit any firmware, log or script contents.
```

Part 5 — Tell me it's ready

You're nearly done. Pick one:

- **Easiest:** email me the summary Claude wrote. I'll take it from there.
- **Or open a pull request yourself:** click the link Claude showed you (or go to the repo on GitHub, where a yellow banner says your branch had recent pushes). Click **Compare & pull request**, check that the base branch says **main**, paste Claude's summary into the description and click **Create pull request**. Don't click Merge; I'll review and merge it.

Sending more files later

Got new logs, a new script or another firmware version after this? Open Claude Code in the same repo folder and paste:

PROMPT 4 — only if you have more to add later

```
I want to add more files to my existing branch phd/[MY FIRST NAME]-<date> (not main).
Same rules as before: copy exactly, change nothing in my folders, edit no firmware
or scripts. Pi logs go in logs/pi/, scripts in tools/pi_log_analysis/ (update
its README for any new or changed script).
1. git fetch origin, then switch to my branch and confirm it is up to date with
   origin. If my supervisor has added commits to it, pull them first.
2. New items: [PATH(S) TO NEW FIRMWARE / PI LOGS / SCRIPTS]
3. Show me a plan as before (commit order and messages, flagged files) and wait
   for my OK. Then commit, verify the checksums, push, and give me the summary.
```

If you already opened a pull request, it updates automatically when you push. If I've already merged your branch, Claude should start a fresh branch from the current main instead. Tell it so if you know.

If something goes wrong

What you see	What to do
"Permission denied" or 403 on push	You haven't accepted the invitation yet, or you signed in as the wrong GitHub account. Accept it (Step 1) and ask Claude to "clear the stored GitHub credentials and push again."
"File exceeds GitHub's file size limit of 100 MB"	Ask Claude: "undo the last commit without losing files, leave the big file out, commit again, push." Then tell me about the file. I'll get it another way.
The clone is very slow or stops partway	The repo is large. Ask Claude to delete the half-finished folder and clone again. It's safe.
Claude starts changing firmware code	Stop it (Esc in the terminal, or the stop button) and say: "Undo that. This task is copy-only. Restore the files to my versions exactly."
You're not sure what state things are in	Ask Claude: "Explain in plain English what branch I'm on, what's committed, and what's been pushed." Asking never changes anything.

You can't damage main by following this guide. The worst outcome is a messy branch, and I can tidy that during review.

Appendix — Fallback without Claude Code (website only)

Use this only if you can't run Claude Code. It skips the per-version history: you get one upload per version and no starting-point check. Regular Claude chat can still walk you through the clicks if you get stuck.

- On the repo page, click the branch selector (it says **main**), type `phd/yourname-date` and choose **Create branch … from main**.
- With **your branch** selected, open the `teensy_controller` folder, then click **Add file → Upload files**. Drag in your **oldest** version's .ino and .h files (same names, so they replace the old ones). Describe the version in the commit box and choose **Commit directly to the phd/… branch**. Repeat for each newer version, in order.
- Upload the Pi logs into `logs/pi/from_yourname_date/` and the scripts into `tools/pi_log_analysis/` the same way. To create a new folder on the website, type the folder name followed by / in the file-name box of **Add file → Create new file**. The website takes at most 25 MB per file and 100 files per upload, so send bigger logs to me another way.
- Email me the branch name, plus which firmware version your work started from.